314540048 - Homework 1

Part A: Basic Questions

Program 1: Find last index of element

A) The loop condition is i > 0 and this prevents the program from checking the index at 0. The function will incorrectly return -1 if the last occurrence of y is at 0.
B) The input validation throws an exception before the faulty loop is even reached.
C) The faulty loop is valuated but the program finds 3 at index 2 and returns immediately. There is no incorrect state that occurs.
D) The faulty loop terminates at i = 0, skipping the final array element. The corrected loop would also examine index 0. This means the control flow is incorrect, but the result is still correct because 4 is not in the array.
E) At the point where the faulty loop exists prematurely:

State	Faulty Programs	Correct Programs
x	[1, 2, 3]	[1, 2, 3]
y	4	4
i	0	0
Control flow	Exits loop	Enters loop body
Final return	-1	-1

314540048 - Homework 1

Code Execution:

```
user@DESKTOP-7QU46KG MINGW64 ~/Desktop/NYCU/Courses/Fall 2026/Software Testing/Homework
$ node mine_findLast.test.js

(a) Explain the fault:
Input: findLast([2, 3, 5], 2)
Expected: 0
Actual: -1
Faulty loop condition (0 > 0): false
Correct loop condition (0 >= 0): true

(b) Fault not executed:
Input: findLast("hello", 2)
Expected: TypeError
Actual: TypeError

(c) Fault executed, no error state:
Input: findLast([1, 2, 3], 3)
Expected: 2
Actual: 2

(d) Error state without failure:
Input: findLast([1, 2, 3], 4)
Expected: -1
Actual: -1
Faulty loop stops when i = 0.
Corrected loop would check index 0.
Both versions return -1.
Note: divergence is in control flow.

(e) First error state:
x = [1, 2, 3]
y = 4
i = 0
Faulty control flow: exits loop.
Correct control flow: enters loop.
Expected final output: -1
Actual final output: -1
First divergence: control-flow location.

Original failing test:
Input: findLast([2, 3, 5], 2)
Expected: 0
Actual: -1

Test execution complete.
```

314540048 - Homework 1

Program 2: Find last index of zero

A) The function scans from the beginning of the array and returns the index of the first zero, rather than the last zero.
B) The array is empty, so the loop body containing the faulty return statement is never executed. The input validation throws an exception before the faulty loop is even reached.
C) The function returns the first zero, which happens to also be the last zero. Therefore, the output is correct.
D) Not possible for ordinary numeric arrays. If the array contains multiple zeros, returning the first zero produces the wrong index and therefore a failure. If it contains only one zero, returning that zero is correct.
E) NA because no qualifying test was identified in (d).

Code Execution:

```
user@DESKTOP-7QU46KG MINGW64 ~/Desktop/NYCU/Courses/Fall 2026/Software Testing/Homework
$ node mine_lastZero.test.js

(a) Explain the fault:
Input: lastZero([0, 1, 0])
Expected: 2
Actual: 0
Faulty loop starts at index: 0
Corrected loop starts at index: 2

(b) Fault not executed:
Input: lastZero([])
Expected: -1
Actual: -1

(c) Fault executed, no error state:
Input: lastZero([0, 1, 2])
Expected: 0
Actual: 0

(d) Error state without failure:
Not possible under the return-value interpretation.
Multiple zeros produce an incorrect index.
One zero produces the correct index.

(e) First error state:
Not applicable because (d) has no qualifying test.

Original failing test:
Input: lastZero([0, 1, 0])
Expected: 2
Actual: 0

Test execution complete.
```

314540048 - Homework 1

Program 3: Count positive elements

A) The condition x[i] >= 0 incorrectly counts zero as a positive number. Positive numbers must be strictly greater than zero.
B) The array is empty, so the loop and faulty condition are never executed.
C) The faulty condition is evaluated, but the array does not contain zero.Therefore, the count remains correct.
D) Not possible for ordinary numeric arrays. Whenever the array contains zero, the faulty condition increments the count incorrectly.
E) Not applicable because (d) is impossible.

Variable	**Faulty Program**	**Correct Program**
x	[-4, 2, 0, 2]	[-4, 2, 0, 2]
y	2	2
count	2	2

314540048 - Homework 1

Code Execution:

```
user@DESKTOP-7QU46KG MINGW64 ~/Desktop/NYCU/Courses/Fall 2026/Software Testing/Homework
$ node mine_countPositives.test.js

(a) Explain the fault:
Input: 0
Faulty condition (0 >= 0): true
Correct condition (0 > 0): false

(b) Fault not executed:
Input: countPositive([])
Expected: 0
Actual: 0

(c) Fault executed, no error state:
Input: countPositive([1, -2, 3])
Expected: 2
Actual: 2

(d) Error state without failure:
Not possible for ordinary numeric arrays.
Zero is incorrectly counted as positive.
This produces an incorrect final result.

(e) First error state:
Not applicable to (d).
Example from the original failing test:
x = [-4, 2, 0, 2]
i = 2
x[i] = 0
Expected count after index 2: 1
Actual count after index 2: 2

Original failing test:
Input: countPositive([-4, 2, 0, 2])
Expected: 2
Actual: 3

Test execution complete.
```

314540048 - Homework 1

Program 4: Count odd or positive elements

A) JavaScript's remainder operator % returns a negative remainder for negative odd numbers.
B) The array is empty, so the loop and faulty condition are never executed.
C) The faulty expression is evaluated, but there are no negative odd numbers. The correct values are counted.
D) Not possible for ordinary integer arrays. Whenever a negative odd number is processed, the faulty condition fails to count it. So the final count is incorrect.
E) Not applicable because no qualifying test case exists for (d).

Variable	Faulty Program	Correct Program
x	[-3, -2, 0, 1, 4]	[-3, -2, 0, 1, 4]
y	0	0
count	0	1

314540048 - Homework 1

Code Execution:

```
user@DESKTOP-7QU46KG MINGW64 ~/Desktop/NYCU/Courses/Fall 2026/Software Testing/Homework
$ node mine_oddOrPos.test.js

(a) Explain the fault:
Input: -3
Remainder (-3 % 2): -1
Faulty condition (-3 % 2 === 1): false
Correct condition (-3 % 2 !== 0): true

(b) Fault not executed:
Input: oddOrPos([])
Expected: 0
Actual: 0

(c) Fault executed, no error state:
Input: oddOrPos([1, 2, -2])
Expected: 2
Actual: 2

(d) Error state without failure:
Not possible for ordinary integer arrays.
This produces an incorrect final result.

(e) First error state:
Not applicable to (d).
Example from the original failing test:
x = [-3, -2, 0, 1, 4]
i = 0
x[i] = -3
Expected count after index 0: 1
Actual count after index 0: 0

Original failing test:
Input: oddOrPos([-3, -2, 0, 1, 4])
Expected: 3
Actual: 2

Test execution complete.
```